Embedded Finance-as-a-Service

(EFaaS) API Platform

Project Architecture, Tech Stack & Build Plan

Portfolio Project for FinTech / Banking / Investment Roles

Tech Stack: Java 17 | Spring Boot 3.x | Kafka | PostgreSQL | Redis | TigerGraph | Docker
External APIs: Stripe | Plaid | Alpaca

Duration: 8 Weeks | Complexity: Advanced

Table of Contents

Table of Contents 2
1. Project Overview 5
1.1 What is EFaaS? 5
1.2 Why This Project? 5
1.3 Target Audience 5
2. System Architecture 6
2.1 High-Level Architecture 6
2.2 Kafka Event-Driven Flow 6
2.3 TigerGraph Fraud Detection Schema 8
2.4 CI/CD Pipeline 8
3. Complete Tech Stack 10
3.1 Core Technologies 10
3.2 External API Integrations 10
4. Microservice Details 12
4.1 API Gateway 12
4.2 Tenant Management Service 12
4.3 Payments Service 12
4.4 Lending Service 12
4.5 Investment Service 13
4.6 Fraud & Risk Intelligence Service 13
5. Week-by-Week Build Plan 14
Goals for This Week 14
Tasks & Checklist 14
What to Learn First 14
Suggested Claude Code Prompts 15
Milestone Check 15
Deliverables 15
Goals for This Week 16
Tasks & Checklist 16
What to Learn First 16
Suggested Claude Code Prompts 17
Milestone Check 17
Deliverables 17
Goals for This Week 18

Tasks & Checklist 18
What to Learn First 18
Suggested Claude Code Prompts 19
Milestone Check 19
Deliverables 19
Goals for This Week 20
Tasks & Checklist 20
What to Learn First 20
Suggested Claude Code Prompts 20
Milestone Check 21
Deliverables 21
Goals for This Week 22
Tasks & Checklist 22
What to Learn First 22
Suggested Claude Code Prompts 23
Milestone Check 23
Deliverables 23
Goals for This Week 24
Tasks & Checklist 24
What to Learn First 24
Suggested Claude Code Prompts 25
Milestone Check 25
Deliverables 25
Goals for This Week 26
Tasks & Checklist 26
What to Learn First 26
Suggested Claude Code Prompts 27
Milestone Check 27
Deliverables 27
Goals for This Week 28
Tasks & Checklist 28
What to Learn First 28
Suggested Claude Code Prompts 29
Milestone Check 29
Deliverables 29
6. Repository Structure 30

7. Interview Talking Points 31
7.1 The Elevator Pitch 31
7.2 Key Technical Discussion Points 31
7.3 Demonstrable Skills 31
8. Future Enhancements (Phase 2) 32

1. Project Overview

1.1 What is EFaaS?

The Embedded Finance-as-a-Service (EFaaS) API Platform is an advanced portfolio project designed to showcase enterprise-grade fintech backend engineering. Instead of building a consumer-facing app, this project builds the infrastructure layer that businesses would use to embed financial services (payments, lending, investments) into their own products.

This is the exact architecture pattern used by companies like Stripe, Marqeta, and Unit. By building the provider side rather than the consumer side, this project demonstrates architectural maturity that typical portfolio projects lack.

1.2 Why This Project?

- **Multi-tenant architecture:** Every API call is scoped to a tenant with API key auth, rate limiting, and data isolation — the same pattern behind Stripe and Plaid.
- **Event-driven microservices:** Services communicate through Kafka events, enabling real cross-service workflows (payment triggers loan repayment match, trade triggers portfolio update).
- **Graph-powered fraud detection:** TigerGraph knowledge graph for fraud ring detection and AML pattern analysis — the exact technology used by JP Morgan and Mastercard.
- **Real fintech API integrations:** Stripe, Plaid, and Alpaca sandbox environments provide production-realistic behavior.
- **Polyglot-ready:** Java core services with a future Python LLM microservice slot, demonstrating polyglot microservices thinking.

1.3 Target Audience

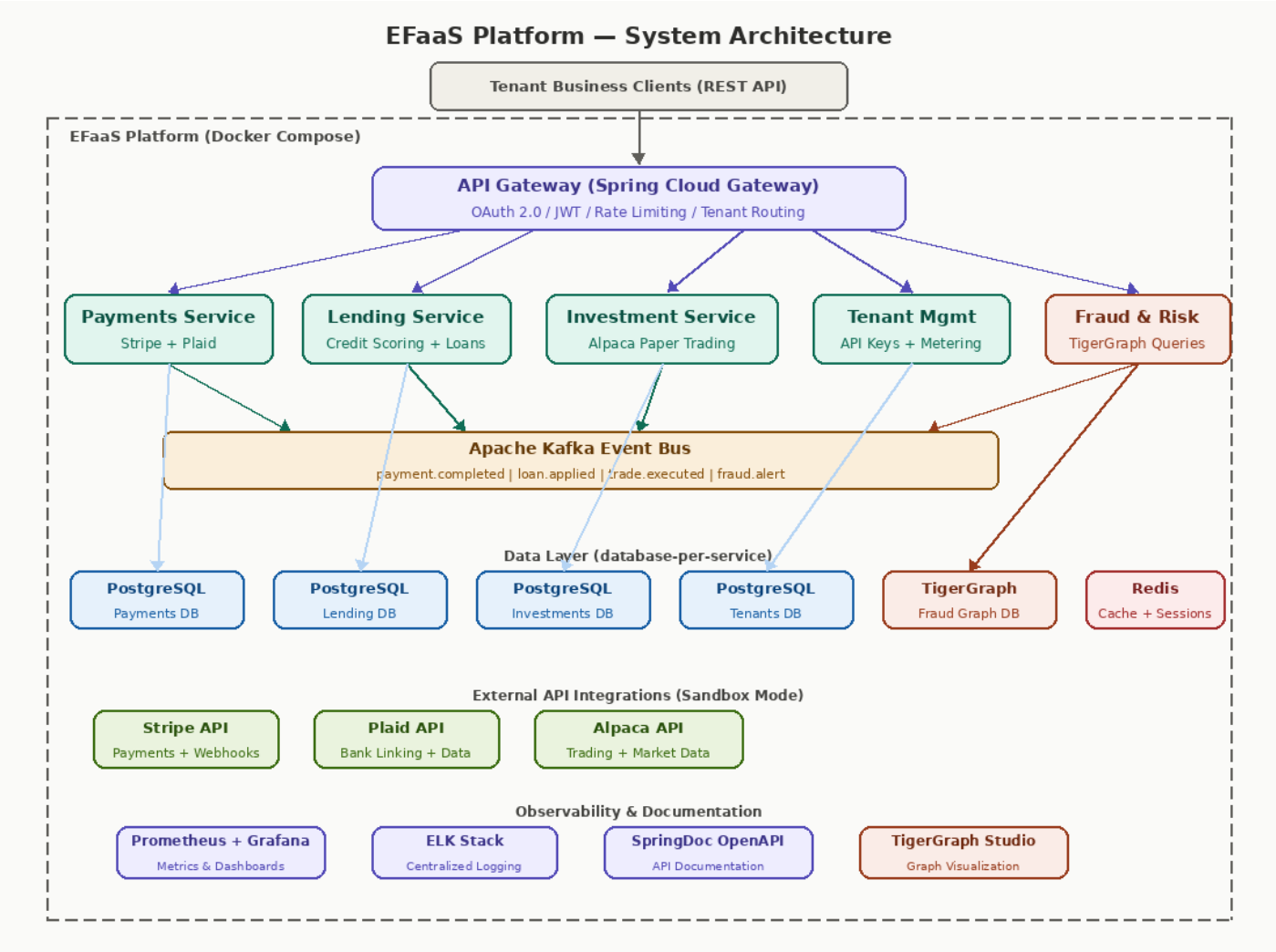
This project is designed to impress recruiters and hiring managers at:

- Major banks (JPMorgan, Goldman Sachs, Morgan Stanley, Barclays)
- Fintech companies (Stripe, Plaid, Square, Robinhood, SoFi, Affirm)
- Investment firms and hedge funds
- Banking-as-a-Service providers (Unit, Marqeta, Galileo)

2. System Architecture

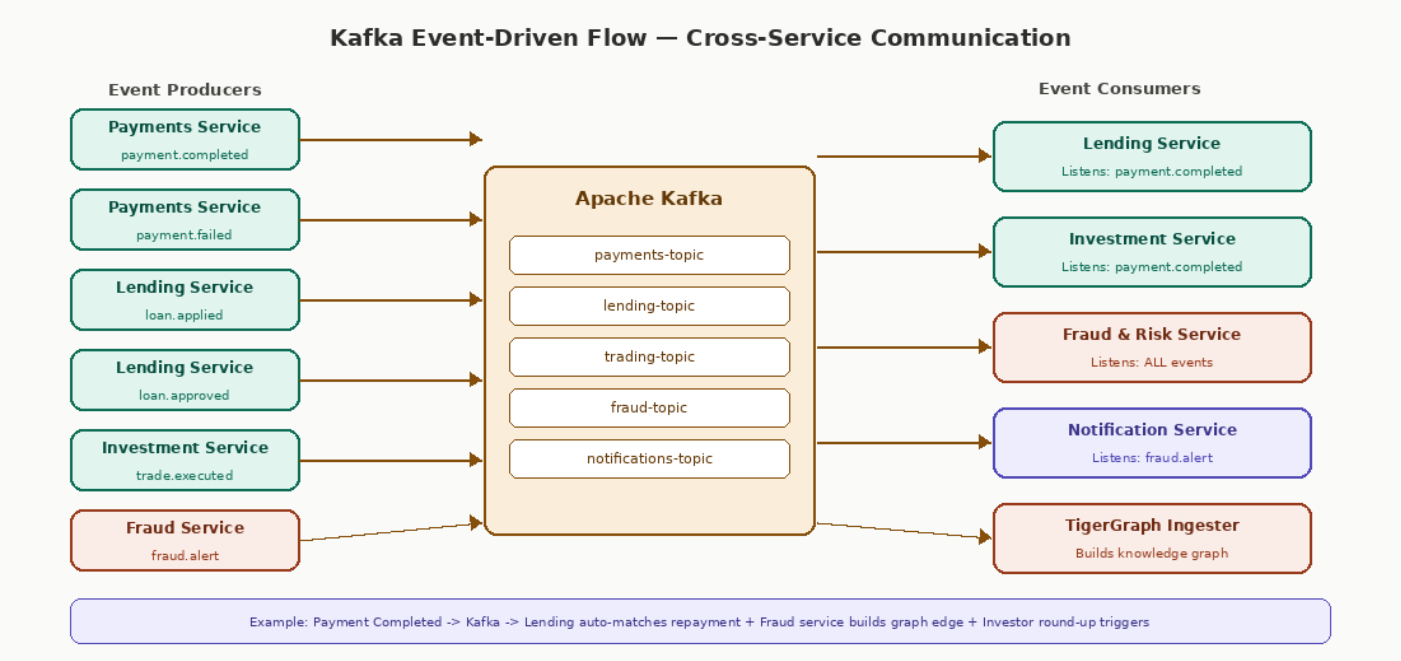
2.1 High-Level Architecture

The platform consists of 5 microservices, an API gateway, an event bus, and multiple data stores. Each service owns its database (database-per-service pattern) and communicates asynchronously through Kafka events.



2.2 Kafka Event-Driven Flow

All cross-service communication happens through Apache Kafka. Each service publishes domain events to specific topics, and interested services subscribe. The Fraud & Risk service subscribes to ALL topics to build a comprehensive knowledge graph.

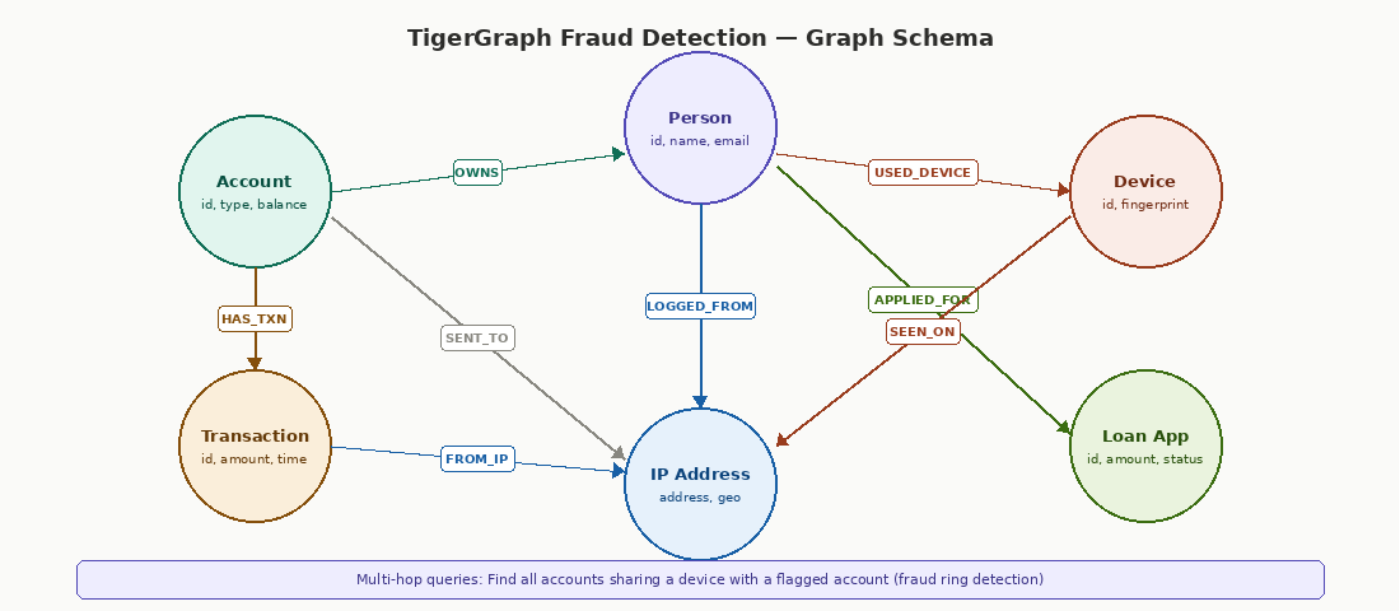


Key event flows in the system:

- **Payment Completed → Lending:** Auto-matches the payment against outstanding loan repayments.
- **Payment Completed → Investments:** Triggers round-up spare change auto-invest into portfolio.
- **Payment Completed → Fraud:** Creates transaction vertex and edges in TigerGraph.
- **Loan Applied → Fraud:** Creates applicant vertices and checks for shared-device fraud patterns.
- **Trade Executed → Fraud:** Monitors for wash trading or unusual velocity patterns.
- **Fraud Alert → All Services:** Can trigger payment holds, loan freezes, or trading suspension.

2.3 TigerGraph Fraud Detection Schema

The Fraud & Risk Intelligence service uses TigerGraph to build a knowledge graph of all entities and relationships across the platform. This enables multi-hop graph queries that detect fraud patterns invisible to traditional SQL.

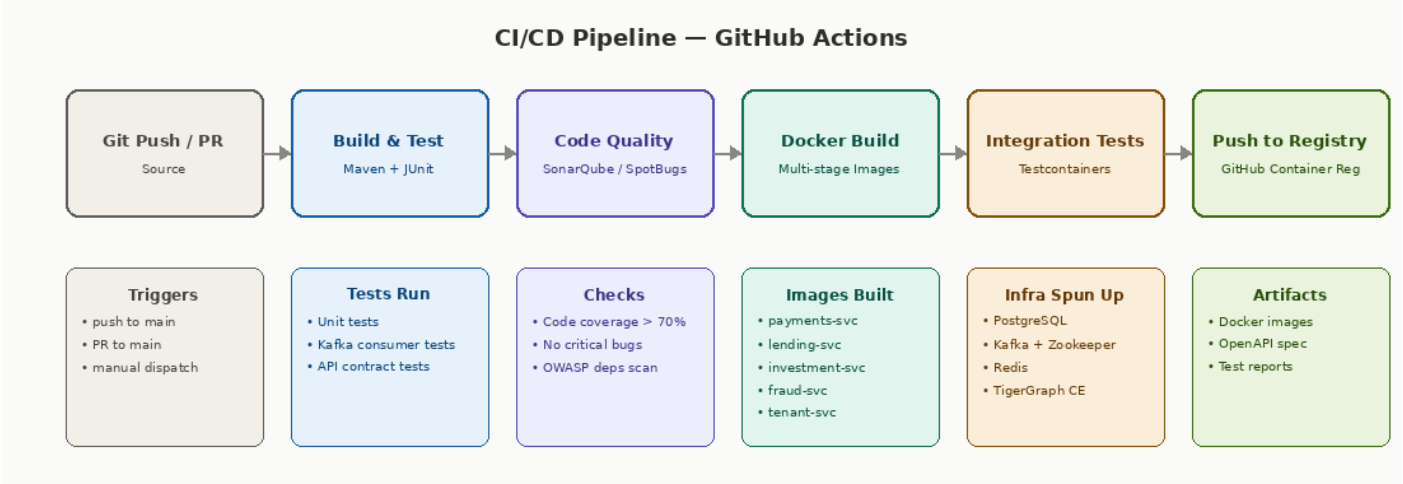


Example GSQL queries for fraud detection:

- **Fraud Ring Detection:** Starting from a flagged account, traverse 3 hops through USED_DEVICE and OWNS edges to find all accounts sharing devices with the flagged account.
- **Velocity Check:** Count transactions from an account within a sliding time window; flag accounts exceeding threshold velocity.
- **Shared Infrastructure Cluster:** Find clusters of accounts that share 2+ attributes (device, IP, email domain) — indicative of synthetic identity fraud.

2.4 CI/CD Pipeline

The project uses GitHub Actions for continuous integration and delivery. Each push triggers a build-test-scan-containerize pipeline.


CI/CD Pipeline — GitHub Actions
Git Push / PR
Source
Build & Test
Maven + JUnit
Code Quality
SonarQube / SpotBugs
Docker Build
Multi-stage Images
Integration Tests
Testcontainers
Push to Registry
GitHub Container Reg
Triggers
• push to main
• PR to main
• manual dispatch
Tests Run
• Unit tests
• Kafka consumer tests
• API contract tests
Checks
• Code coverage > 70%
• No critical bugs
• OWASP deps scan
Images Built
• payments-svc
• lending-svc
• investment-svc
• fraud-svc
• tenant-svc
Infra Spun Up
• PostgreSQL
• Kafka + Zookeeper
• Redis
• TigerGraph CE
Artifacts
• Docker images
• OpenAPI spec
• Test reports

3. Complete Tech Stack

3.1 Core Technologies

Category	Technology	Purpose
Language	Java 17+	Primary backend language; strong enterprise fintech adoption
Framework	Spring Boot 3.x	Microservices framework; dependency injection, auto-config
API Gateway	Spring Cloud Gateway	Request routing, rate limiting, JWT validation, tenant isolation
Security	Spring Security + OAuth 2.0	JWT-based authentication, role-based access, API key validation
Messaging	Apache Kafka	Async event streaming between microservices
Primary DB	PostgreSQL	ACID-compliant relational database (one per service)
Graph DB	TigerGraph CE	Fraud detection knowledge graph, multi-hop relationship queries
Cache	Redis	API response caching, rate limit counters, session storage
Containers	Docker + Compose	Containerization of all services and infrastructure
API Docs	SpringDoc OpenAPI	Auto-generated Swagger/OpenAPI documentation
Build Tool	Maven	Dependency management, multi-module project build
CI/CD	GitHub Actions	Automated build, test, and container image pipeline
Monitoring	Prometheus + Grafana	Metrics collection and real-time dashboards
Logging	ELK Stack / Logback	Centralized structured logging
Testing	JUnit 5 + Testcontainers	Unit tests + integration tests with real containers

3.2 External API Integrations

API	Used In	Features Used	Environment
Stripe	Payments Service	Payment intents, ACH transfers, webhooks, refunds	Test/Sandbox mode

Plaid	Payments + Lending	Bank linking (Link), balance checks, transaction history, income verification	Sandbox (user_good / pass_good)
Alpaca	Investment Service	Paper trading API, market/limit/stop orders, portfolio history, real-time market data WebSocket	Paper trading environment

4. Microservice Details

4.1 API Gateway

Technology	Spring Cloud Gateway
Port	8080
Responsibilities	Request routing, JWT validation, API key authentication, rate limiting (Redis-backed), tenant context injection, request/response logging
Key Endpoints	POST /auth/token, GET /health, All /api/v1/* routes proxied to services

4.2 Tenant Management Service

Technology	Spring Boot 3.x + PostgreSQL
Port	8081
Responsibilities	Tenant CRUD, API key generation/rotation, usage metering, plan/tier management, onboarding workflow
Key Endpoints	POST /api/v1/tenants, POST /api/v1/tenants/{id}/api-keys, GET /api/v1/tenants/{id}/usage, PUT /api/v1/tenants/{id}/plan
Database Tables	tenants, api_keys, usage_records, plans

4.3 Payments Service

Technology	Spring Boot 3.x + PostgreSQL + Stripe SDK + Plaid SDK
Port	8082
Stripe Features	Payment Intents (card/ACH), refunds, webhook handlers (payment_intent.succeeded, charge.refunded), idempotency keys
Plaid Features	Plaid Link token creation, bank account linking, balance checks, transaction history retrieval (90 days)
Kafka Events	Publishes: payment.completed, payment.failed, refund.processed
Key Endpoints	POST /api/v1/payments, POST /api/v1/payments/{id}/refund, POST /api/v1/bank-accounts/link, GET /api/v1/bank-accounts/{id}/balance
Database Tables	payments, refunds, bank_accounts, payment_methods, webhook_events

4.4 Lending Service

Technology	Spring Boot 3.x + PostgreSQL
Port	8083

Credit Scoring	Rule-based engine: debt-to-income ratio, payment history score (from Plaid data), balance stability, employment duration. Returns score 300-850 with risk tier.
Loan Lifecycle	Application → Scoring → Approval/Denial → Disbursement (via Stripe) → Repayment Schedule → Payment Tracking → Completion/Default
Kafka Events	Publishes: loan.applied, loan.approved, loan.denied, loan.disbursed. Consumes: payment.completed (auto-match repayments)
Key Endpoints	POST /api/v1/loans/apply, GET /api/v1/loans/{id}, GET /api/v1/loans/{id}/schedule, POST /api/v1/loans/{id}/disburse
Database Tables	loan_applications, loans, repayment_schedules, credit_scores

4.5 Investment Service

Technology	Spring Boot 3.x + PostgreSQL + Alpaca SDK + WebSocket
Port	8084
Alpaca Features	Paper trading: market/limit/stop orders, portfolio positions, portfolio history, real-time price streaming via WebSocket
Auto-Invest	Listens for payment.completed events; rounds up to nearest dollar; invests spare change into configured portfolio (e.g., SPY ETF)
Kafka Events	Publishes: trade.executed, portfolio.rebalanced. Consumes: payment.completed (round-up auto-invest)
Key Endpoints	POST /api/v1/orders, GET /api/v1/portfolio, GET /api/v1/portfolio/history, GET /api/v1/market/quotes/{symbol}

4.6 Fraud & Risk Intelligence Service

Technology	Spring Boot 3.x + TigerGraph Community Edition + REST API
Port	8085 (service) + 14240 (GraphStudio) + 9000 (TG REST)
Graph Schema	Vertices: Account, Person, Device, IPAddress, Transaction, LoanApplication. Edges: OWNS, USED_DEVICE, LOGGED_FROM, SENT_TO, APPLIED_FOR, FROM_IP, SEEN_ON
GSQL Queries	fraud_ring_detection (multi-hop device sharing), velocity_check (transaction frequency), shared_infrastructure_cluster (2+ shared attributes), risk_score (composite scoring)
Kafka Events	Consumes: ALL topics (payment.*, loan.*, trade.*). Publishes: fraud.alert
Key Endpoints	GET /api/v1/risk/score/{accountId}, GET /api/v1/risk/fraud-ring/{accountId}, POST /api/v1/risk/check-transaction

5. Week-by-Week Build Plan

Each week includes specific goals, a task checklist, learning resources, Claude Code prompts for assistance, and a milestone to verify before moving on. Check off tasks as you complete them.

WEEK 1: Project Foundation & Tenant Service

Goals for This Week

- Set up the multi-module Maven project structure with parent POM
- Configure Docker Compose with PostgreSQL, Kafka, Redis, and TigerGraph
- Build the Tenant Management microservice (CRUD + API key generation)
- Set up Spring Cloud Gateway with basic routing and JWT skeleton
- Initialize Git repo with proper .gitignore and README skeleton

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Create multi-module Maven project (parent + 6 modules: gateway, tenant-svc, payment-svc, lending-svc, investment-svc, fraud-svc)	*Create a Maven multi-module Spring Boot 3.x project for a fintech microservices platform with 6 child modules*
☐	Write docker-compose.yml with PostgreSQL (5 instances), Kafka + Zookeeper, Redis, TigerGraph CE	*Write a docker-compose.yml with 5 PostgreSQL instances, Kafka with Zookeeper, Redis, and TigerGraph Community Edition*
☐	Implement Tenant entity, repository, service, and REST controller	*Create a Spring Boot CRUD service for tenant management with JPA, PostgreSQL, and proper DTOs*
☐	Implement API key generation (UUID + SHA-256 hash storage)	*Implement secure API key generation with SHA-256 hashing in Spring Boot*
☐	Configure Spring Cloud Gateway routes to each microservice	*Configure Spring Cloud Gateway to route /api/v1/payments/** to payment service on port 8082*
☐	Set up Spring Security with JWT token validation	*Set up Spring Security with JWT validation for a Spring Cloud Gateway*
☐	Write basic health check endpoints for all services	
☐	Create GitHub repo, .gitignore, initial README with project description	
☐	Verify all Docker containers start and communicate	

What to Learn First

Topic	Resource	Time Estimate
Spring Boot 3.x multi-module Maven	Spring.io Guides: Multi-Module Projects	2 hours
Spring Cloud Gateway basics	Baeldung: Spring Cloud Gateway guide	3 hours
Docker Compose networking	Docker Docs: Networking in Compose	1 hour
JWT authentication flow	Baeldung: JWT with Spring Security	2 hours
Kafka concepts (producers/consumers)	Confluent: Apache Kafka 101 (YouTube)	2 hours

Suggested Claude Code Prompts

```
> Create a Spring Boot 3.2 multi-module Maven project with parent POM defining Java 17, Spring Boot BOM, and 6 child modules: gateway, tenant-service, payment-service, lending-service, investment-service, fraud-service

> Write docker-compose.yml with: postgres-tenants (port 5433), postgres-payments (5434), postgres-lending (5435), postgres-investments (5436), kafka + zookeeper, redis, tigergraph/community (ports 14240, 9000)

> Implement TenantController with POST /api/v1/tenants (create), GET /{id}, PUT /{id}, DELETE /{id}, and POST /{id}/api-keys (generate key). Use DTOs, not entities in API responses.

> Add a GlobalExceptionHandler with @ControllerAdvice returning RFC 7807 ProblemDetail responses for all services
```

Milestone Check

> ☐ **MILESTONE:** Docker Compose starts all containers; Tenant CRUD API works through the Gateway; API key can be generated and used to authenticate requests

Deliverables

- Working docker-compose.yml with all infrastructure running
- Tenant Management service with full CRUD + API key generation
- Spring Cloud Gateway routing to tenant service
- JWT authentication skeleton (can issue and validate tokens)
- GitHub repo initialized with README and .gitignore

WEEK 2: Gateway Hardening & Rate Limiting

Goals for This Week

- Implement Redis-backed rate limiting per API key in the gateway
- Add tenant context injection (gateway extracts tenant ID from API key and passes as header)
- Set up Kafka infrastructure with Spring Kafka configuration
- Write integration tests with Testcontainers
- Set up GitHub Actions CI pipeline (build + test)

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Implement Redis-backed rate limiter using Spring Cloud Gateway RequestRateLimiter filter	*Implement Redis-backed rate limiting in Spring Cloud Gateway per API key with configurable rates per tenant plan*
☐	Create API key authentication filter that validates key against tenant service and injects X-Tenant-Id header	*Create a Spring Cloud Gateway GlobalFilter that validates API key, looks up tenant, and adds X-Tenant-Id header*
☐	Configure Spring Kafka producer and consumer beans for all services	*Configure Spring Kafka with JSON serializer/deserializer for domain events in Spring Boot 3.x*
☐	Create base event classes: PaymentEvent, LoanEvent, TradeEvent, FraudEvent	*Create a base Kafka event class hierarchy with PaymentCompletedEvent, LoanAppliedEvent etc. using Java records*
☐	Write Testcontainers integration tests for Tenant service (PostgreSQL + Redis)	*Write integration tests for a Spring Boot JPA service using Testcontainers with PostgreSQL*
☐	Set up GitHub Actions workflow: build, test, cache Maven dependencies	*Create a GitHub Actions CI workflow for a multi-module Maven Spring Boot project*
☐	Add request/response logging filter in gateway	
☐	Add usage metering: gateway logs each API call per tenant to a usage_records table	*Implement async API usage metering in Spring Cloud Gateway that records per-tenant call counts*

What to Learn First

Topic	Resource	Time Estimate
Spring Cloud Gateway filters	Spring Docs: Gateway Filters	2 hours

Redis rate limiting patterns	Baeldung: Rate Limiting with Redis	1.5 hours
Spring Kafka configuration	Baeldung: Spring Kafka Tutorial	3 hours
Testcontainers for Java	Testcontainers.org: Quickstart guide	2 hours
GitHub Actions basics	GitHub Docs: Building Java with Maven	1 hour

Suggested Claude Code Prompts

```
> Implement a Redis-backed rate limiter in Spring Cloud Gateway that reads the rate limit from the tenant's plan tier (free=10/min, basic=100/min, pro=1000/min)

> Write a Spring Kafka configuration class with KafkaTemplate<String, Object> using JsonSerializer and configure consumer groups per service

> Create a Testcontainers-based integration test for TenantService that spins up a real PostgreSQL container

> Create a .github/workflows/ci.yml that runs mvn clean verify on every push and PR to main
```

Milestone Check

> ☐ **MILESTONE:** Gateway authenticates API keys, applies rate limits per tenant plan, injects tenant context, and all Kafka producers/consumers can send and receive test events

Deliverables

- Rate limiting working per API key via Redis
- API key validation filter passing X-Tenant-Id to downstream services
- Spring Kafka configuration with test event round-trip verified
- GitHub Actions CI pipeline running on push/PR
- Integration tests passing with Testcontainers

WEEK 3: Payments Service — Stripe Integration

Goals for This Week

- Integrate Stripe SDK for payment processing in sandbox mode
- Implement payment creation, confirmation, and refund flows
- Build Stripe webhook handler for async payment status updates
- Publish payment events to Kafka on state transitions

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Add Stripe Java SDK dependency and configure with test API key	*Set up Stripe Java SDK in Spring Boot with test mode API key configuration*
☐	Implement POST /api/v1/payments endpoint using Stripe PaymentIntents API	*Implement a payment creation endpoint using Stripe PaymentIntents with idempotency keys in Spring Boot*
☐	Build Stripe webhook endpoint (POST /webhooks/stripe) with signature verification	*Implement a Stripe webhook handler in Spring Boot with signature verification and idempotent event processing*
☐	Handle webhook events: payment_intent.succeeded, payment_intent.payment_failed, charge.refunded	
☐	Implement POST /api/v1/payments/{id}/refund endpoint	
☐	Publish PaymentCompletedEvent and PaymentFailedEvent to Kafka payments-topic	*Publish Kafka events from a Spring Boot service when Stripe webhook confirms payment success or failure*
☐	Store payment records with status tracking in PostgreSQL	
☐	Add idempotency key support to prevent duplicate payments	
☐	Write unit tests for payment service logic and webhook handler	

What to Learn First

Topic	Resource	Time Estimate
Stripe API fundamentals	Stripe Docs: Accept a payment (Java)	3 hours
Stripe webhooks	Stripe Docs: Webhook endpoints	2 hours
Stripe test mode	Stripe Docs: Testing (use test card 4242424242424242)	1 hour

Idempotency in payments	Stripe Docs: Idempotent Requests	1 hour

Suggested Claude Code Prompts

```
> Integrate Stripe PaymentIntents API in Spring Boot: create payment, handle async confirmation via webhook, store state transitions in PostgreSQL
> Build a Stripe webhook handler that verifies signatures, processes payment_intent.succeeded events, and publishes PaymentCompletedEvent to Kafka
> Write JUnit tests for PaymentService mocking Stripe API calls with test fixtures
```

Milestone Check

> ☐ **MILESTONE:** Can create a payment via API, receive Stripe webhook confirmation, see payment status update in DB, and verify Kafka payment.completed event is published

Deliverables

- Payment creation endpoint working with Stripe sandbox
- Webhook handler processing Stripe events with signature verification
- Refund endpoint functional
- Kafka payment events publishing on state changes
- Unit tests covering payment flows

WEEK 4: Payments Service — Plaid Integration

Goals for This Week

- Integrate Plaid SDK for bank account linking in sandbox mode
- Implement Plaid Link token flow for frontend bank connection
- Retrieve bank balances and transaction history via Plaid
- Connect Plaid data to enhance payment and lending capabilities

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Add Plaid Java SDK and configure sandbox credentials	*Set up Plaid Java SDK in Spring Boot with sandbox environment configuration*
☐	Implement POST /api/v1/bank-accounts/link-token to create Plaid Link token	*Implement Plaid Link token creation endpoint in Spring Boot for bank account connection*
☐	Implement POST /api/v1/bank-accounts/exchange-token to exchange public_token for access_token	
☐	Implement GET /api/v1/bank-accounts/{id}/balance to fetch real-time balance from Plaid	
☐	Implement GET /api/v1/bank-accounts/{id}/transactions to fetch 90-day transaction history	*Fetch transaction history from Plaid API in Spring Boot and store in PostgreSQL*
☐	Store Plaid access tokens securely (encrypted at rest)	
☐	Create a shared FinancialProfile DTO that Lending service will consume for credit scoring	
☐	Write integration tests using Plaid sandbox credentials (user_good / pass_good)	

What to Learn First

Topic	Resource	Time Estimate
Plaid API fundamentals	Plaid Docs: Quickstart guide	3 hours
Plaid Link flow	Plaid Docs: Link Token creation	1.5 hours
Plaid Sandbox	Plaid Docs: Sandbox test credentials	30 min
Encrypting secrets at rest	Baeldung: Jasypt encryption in Spring Boot	1 hour

Suggested Claude Code Prompts

```
> Implement the full Plaid Link flow: create link token, exchange public
token for access token, retrieve accounts list, fetch balances and
transactions
> Create a FinancialProfileService that aggregates Plaid data (avg
balance, monthly income, payment regularity) into a FinancialProfile DTO
for credit scoring
```

Milestone Check

> ☐ **MILESTONE:** Can link a sandbox bank account via Plaid, retrieve balance and transactions, and the FinancialProfile DTO is ready for the Lending service to consume

Deliverables

- Plaid Link token creation and token exchange working
- Balance and transaction retrieval endpoints functional
- FinancialProfile aggregation DTO ready
- Access tokens stored securely
- Integration tests passing with Plaid sandbox

WEEK 5: Lending Service — Credit Scoring & Loan Lifecycle

Goals for This Week

- Build the rule-based credit scoring engine consuming Plaid financial profile data
- Implement the full loan application and approval workflow
- Create repayment schedule generation and payment tracking
- Subscribe to Kafka payment events for auto-matching loan repayments

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Implement CreditScoringEngine with weighted rules: DTI ratio (30%), payment history (25%), balance stability (20%), account age (15%), income level (10%)	*Implement a rule-based credit scoring engine in Java with weighted factors: debt-to-income, payment history, balance stability, account age, income level*
☐	Create POST /api/v1/loans/apply endpoint that triggers credit scoring	
☐	Implement approval logic: score >= 700 auto-approve, 500-699 conditional, < 500 deny	
☐	Generate amortization schedule (monthly payments, interest calculation) on loan approval	*Generate a loan amortization schedule with principal, interest, and remaining balance calculations in Java*
☐	Implement loan disbursement endpoint that triggers Stripe transfer	
☐	Build Kafka consumer for payment.completed events to auto-match repayments against schedule	*Create a Kafka consumer that listens for payment.completed events and auto-matches them against outstanding loan repayment schedules*
☐	Publish loan.applied, loan.approved, loan.denied events to Kafka	
☐	Add overdue detection: scheduled job checks for missed payments	
☐	Write unit tests for credit scoring engine with various financial profiles	

What to Learn First

Topic	Resource	Time Estimate
Credit scoring fundamentals	Investopedia: How Credit Scores Work	1 hour
Loan amortization math	Khan Academy: Amortization Schedules	1 hour

Kafka consumer groups	Confluent: Kafka Consumer Guide	2 hours
Spring @Scheduled tasks	Baeldung: Scheduling in Spring	1 hour

Suggested Claude Code Prompts

```
> Build a CreditScoringEngine class with these weighted factors and scoring rules: DTI < 0.36 = 200pts, 0.36-0.50 = 100pts, > 0.50 = 0pts (similar breakdowns per factor)

> Create a Kafka consumer in the lending service that matches payment.completed events to the next outstanding installment in the repayment schedule

> Generate a loan amortization schedule: given principal, annual rate, and term in months, calculate monthly payment amount and produce a list of (month, payment, principal, interest, balance) records
```

Milestone Check

> ☐ **MILESTONE:** A loan application flows through credit scoring, gets approved/denied, generates a repayment schedule, and incoming payments are automatically matched to installments via Kafka events

Deliverables

- Credit scoring engine with 5 weighted factors
- Loan application → scoring → approval/denial workflow
- Amortization schedule generation
- Kafka consumer auto-matching payments to repayments
- Overdue detection scheduled job
- Unit tests for scoring engine with edge cases

WEEK 6: Investment Service — Alpaca Trading

Goals for This Week

- Integrate Alpaca paper trading API for order execution
- Build portfolio tracking with positions and history endpoints
- Implement round-up auto-invest feature via Kafka payment events
- Set up real-time market data streaming via WebSocket

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Add Alpaca Java SDK and configure paper trading API keys	*Set up Alpaca Markets Java SDK for paper trading in Spring Boot*
☐	Implement POST /api/v1/orders endpoint (market, limit, stop order types)	*Implement a trading order endpoint that supports market, limit, and stop orders via Alpaca API in Spring Boot*
☐	Implement GET /api/v1/portfolio (positions, total value, gains/losses)	
☐	Implement GET /api/v1/portfolio/history (daily portfolio value over time)	
☐	Build WebSocket endpoint for real-time price streaming from Alpaca	*Create a Spring WebSocket handler that streams real-time market data from Alpaca to connected clients*
☐	Build Kafka consumer for payment.completed: calculate round-up, place auto-invest order	*Create a Kafka consumer that rounds up payment amounts to the nearest dollar and invests the difference in SPY ETF via Alpaca*
☐	Publish trade.executed events to Kafka	
☐	Add basic risk management: max position size check, daily loss limit	
☐	Write tests using Alpaca paper trading (no real money)	

What to Learn First

Topic	Resource	Time Estimate
Alpaca Trading API	Alpaca Docs: Trading API guide	3 hours
Alpaca Paper Trading	Alpaca Docs: Paper Trading setup	1 hour
Spring WebSocket	Baeldung: WebSockets with Spring	2 hours

Order types (market/limit/stop)	Investopedia: Stock Order Types	1 hour

Suggested Claude Code Prompts

```
> Implement an order placement endpoint that validates order type (MARKET/LIMIT/STOP), checks position size limits, calls Alpaca API, stores the order, and publishes trade.executed to Kafka

> Create a Kafka consumer that listens for payment.completed events, calculates the round-up to the nearest dollar, and places a fractional share buy order for SPY via Alpaca paper trading API

> Build a WebSocket endpoint at /ws/market-data that connects to Alpaca's streaming API and broadcasts real-time quotes to subscribed clients
```

Milestone Check

> ☐ **MILESTONE:** Can place paper trading orders via API, see portfolio positions and history, receive real-time price updates via WebSocket, and round-up auto-invest is triggered by payment events

Deliverables

- Order placement endpoint (market/limit/stop) via Alpaca
- Portfolio tracking (positions, history, P&L)
- Real-time market data WebSocket streaming
- Round-up auto-invest via Kafka payment events
- Basic risk management checks

WEEK 7: Fraud & Risk Service — TigerGraph Integration

Goals for This Week

- Set up TigerGraph Community Edition schema with GSQL
- Build Kafka consumer that ingests all events and populates the graph
- Write GSQL queries for fraud ring detection and risk scoring
- Expose fraud detection APIs through the gateway

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Design graph schema in GraphStudio: 6 vertex types, 7 edge types (see Architecture section)	*Write a GSQL schema for fraud detection with vertices: Account, Person, Device, IPAddress, Transaction, LoanApplication and edges: OWNS, USED_DEVICE, LOGGED_FROM, SENT_TO, APPLIED_FOR, FROM_IP, SEEN_ON*
☐	Write GSQL loading job to seed initial test data	
☐	Build Kafka consumer that transforms events into TigerGraph REST API upsert calls	*Create a Spring Kafka consumer that transforms payment/lending/trading events into TigerGraph vertex and edge upserts via REST API*
☐	Write GSQL query: fraud_ring_detection (3-hop traversal from flagged account via shared devices)	*Write a GSQL installed query that starting from a given account, traverses 3 hops via USED_DEVICE and OWNS edges to find all connected accounts*
☐	Write GSQL query: velocity_check (count transactions in sliding window)	
☐	Write GSQL query: shared_infrastructure_cluster (accounts sharing 2+ attributes)	
☐	Write GSQL query: composite_risk_score (weighted combination of graph signals)	
☐	Implement REST endpoints: GET /risk/score/{accountId}, GET /risk/fraud-ring/{accountId}	
☐	Add TigerGraph container health check to docker-compose	
☐	Test with seed data: create a synthetic fraud ring and verify detection	

What to Learn First

Topic	Resource	Time Estimate
TigerGraph GraphStudio UI	TigerGraph Docs: Getting Started with Docker	2 hours
GSQL fundamentals	TigerGraph Docs: GSQL 101 tutorial	4 hours
TigerGraph REST API	TigerGraph Docs: REST API Reference	2 hours
Graph-based fraud detection	TigerGraph Blog: Anti-Fraud demo walkthrough	1.5 hours
Multi-hop graph traversal	TigerGraph Docs: ACCUM and traversal	2 hours

Suggested Claude Code Prompts

```
> Write a GSQL schema definition with CREATE VERTEX and CREATE EDGE
statements for: Account (id STRING, type STRING, balance DOUBLE), Person
(id STRING, name STRING, email STRING), Device (id STRING, fingerprint
STRING), IPAddress (address STRING, geo STRING), Transaction (id STRING,
amount DOUBLE, timestamp DATETIME), LoanApplication (id STRING, amount
DOUBLE, status STRING)

> Write a GSQL installed query called fraud_ring_detection that takes an
accountId parameter and traverses 3 hops through USED_DEVICE and OWNS
edges to return all connected accounts with their connection depth

> Create a Spring Boot Kafka consumer that listens to payments-topic,
lending-topic, and trading-topic, transforms each event into TigerGraph
vertex/edge upserts, and calls TigerGraph REST API at
http://tigergraph:9000
```

Milestone Check

> ☐ **MILESTONE:** TigerGraph schema is deployed, Kafka events are populating the graph in real-time, fraud ring detection query correctly identifies connected accounts from seed test data, and risk score API returns a composite score

Deliverables

- TigerGraph schema deployed with 6 vertex and 7 edge types
- Kafka-to-graph ingestion pipeline working
- 3-4 GSQL fraud detection queries installed and tested
- REST APIs for risk scoring and fraud ring detection
- Seed data test proving fraud ring detection works

WEEK 8: Polish, Documentation & Deployment

Goals for This Week

- Add comprehensive OpenAPI/Swagger documentation for all services
- Set up Prometheus metrics and Grafana dashboards
- Write a professional README with architecture diagrams
- Run end-to-end integration tests and fix edge cases
- Record a demo video or write a demo walkthrough

Tasks & Checklist

Done	Task	Claude Code Prompt
☐	Add SpringDoc OpenAPI annotations to all REST controllers	*Add SpringDoc OpenAPI annotations with descriptions, examples, and response codes to all REST endpoints in the payments service*
☐	Configure Prometheus actuator endpoint in each service	*Configure Spring Boot Actuator with Prometheus metrics endpoint and custom business metrics*
☐	Create Grafana dashboard JSON with panels for: request rate, error rate, Kafka lag, payment volume	*Create a Grafana dashboard JSON for monitoring Spring Boot microservices with Prometheus*
☐	Write comprehensive README.md with: project overview, architecture diagrams, getting started (docker-compose up), API examples	*Write a professional GitHub README for a fintech microservices project with architecture diagrams, quick start, and API examples*
☐	Add CONTRIBUTING.md and LICENSE	
☐	Run full end-to-end test: create tenant → link bank → make payment → apply for loan → place trade → check fraud score	
☐	Fix any edge cases found during E2E testing	
☐	Add structured logging with correlation IDs across services	*Implement distributed tracing with correlation IDs across Spring Boot microservices using MDC*
☐	Create docker-compose.prod.yml with resource limits and restart policies	
☐	Write DEMO.md walkthrough showing the key flows with curl examples	
☐	Record a 3-5 minute demo video (optional but impressive)	

What to Learn First

Topic	Resource	Time Estimate
SpringDoc OpenAPI	SpringDoc Docs: Getting Started	2 hours
Prometheus + Grafana setup	Baeldung: Monitoring Spring Boot with Prometheus	2 hours
Writing great READMEs	GitHub Docs: About READMEs + awesome-readme examples	1 hour
Structured logging	Baeldung: Logging with MDC in Spring Boot	1 hour

Suggested Claude Code Prompts

```
> Add @Operation, @ApiResponse, and @Schema annotations to all payment service endpoints with request/response examples

> Write a comprehensive README.md with badges, architecture diagram reference, prerequisites, one-command startup (docker-compose up), and curl examples for each API endpoint

> Create a DEMO.md that walks through: 1) create tenant 2) generate API key 3) link bank via Plaid 4) make payment 5) check loan eligibility 6) place trade 7) check fraud score - with exact curl commands and expected responses
```

Milestone Check

> ☐ **MILESTONE:** Full system runs with docker-compose up. README is professional-grade. Swagger UI shows all endpoints. Grafana dashboard displays live metrics. E2E flow works end-to-end from tenant creation through fraud scoring.

Deliverables

- OpenAPI/Swagger documentation for all services
- Prometheus + Grafana monitoring stack
- Professional README with architecture diagrams
- DEMO.md with curl-based walkthrough
- E2E integration test passing
- Structured logging with correlation IDs
- Clean git history with meaningful commits

6. Repository Structure

The project follows a Maven multi-module layout. Each microservice is an independent Spring Boot application with its own configuration, tests, and Dockerfile.

```
efaas-platform/
├── docker-compose.yml
├── docker-compose.prod.yml
├── pom.xml                    (parent POM)
├── README.md
├── DEMO.md
├── .github/workflows/ci.yml
├── gateway/
│   ├── src/main/java/.../gateway/
│   ├── src/main/resources/application.yml
│   └── Dockerfile
├── tenant-service/
│   ├── src/main/java/.../tenant/
│   ├── src/test/java/.../tenant/
│   └── Dockerfile
├── payment-service/            (same structure)
├── lending-service/
├── investment-service/
├── fraud-service/
│   ├── gsql/                  (GSQL schema + queries)
│   └── seed-data/             (test fraud ring data)
├── common/                     (shared DTOs, events, utils)
├── grafana/                    (dashboard JSON exports)
└── prometheus/                 (prometheus.yml config)
```

7. Interview Talking Points

When presenting this project to recruiters or in technical interviews, frame it around these key narratives:

7.1 The Elevator Pitch

> *"I built a multi-tenant embedded finance platform with microservices for payments, lending, and investments — plus a graph-powered fraud intelligence layer using TigerGraph that detects suspicious patterns across the entire transaction network in real-time. The platform is designed as B2B infrastructure — like a mini Stripe — where businesses integrate via API keys to embed financial services into their own products."*

7.2 Key Technical Discussion Points

- **Why microservices?** Each financial domain (payments, lending, investments) has different scaling patterns and failure modes. Payments need high throughput; lending needs complex business logic; investments need real-time data. Separate services let you scale and deploy independently.
- **Why Kafka over REST for inter-service?** Financial events need guaranteed delivery and ordering. If the fraud service is temporarily down, payment events are retained in Kafka and processed when it recovers. REST calls would lose those events.
- **Why TigerGraph over SQL for fraud?** Fraud ring detection requires multi-hop traversals (find all accounts 3 hops away sharing devices). In SQL, this requires self-joins that grow exponentially. In TigerGraph, it's a simple ACCUM traversal that runs in milliseconds.
- **Why database-per-service?** Each service manages its own schema and migrations independently. If the lending service needs a schema change, it doesn't risk breaking the payments service. This is the standard pattern in production microservices.

7.3 Demonstrable Skills

Skill Category	Evidence in This Project
System Design	Multi-tenant B2B platform with 5 microservices, API gateway, event bus
Java / Spring Boot	Production-grade Spring Boot 3.x with Security, Cloud Gateway, Kafka, JPA
API Design	RESTful APIs with OpenAPI documentation, versioning, idempotency
Event-Driven Architecture	Kafka-based async communication with domain events
Database Design	PostgreSQL (relational) + TigerGraph (graph) + Redis (cache) — polyglot persistence
Financial Domain Knowledge	Payments (Stripe), banking data (Plaid), trading (Alpaca), credit scoring, AML
DevOps / CI-CD	Docker Compose, GitHub Actions, Prometheus + Grafana monitoring

Security	OAuth 2.0, JWT, API key management, rate limiting, encrypted secrets
Testing	JUnit 5, Testcontainers, integration tests with real containers

8. Future Enhancements (Phase 2)

After the initial 8-week build, these enhancements can be added to further strengthen the project:

- **LLM Financial Insights Service (Python):** A Python microservice that consumes Kafka events, analyzes transaction patterns via an LLM, and returns natural-language risk summaries and spending insights. Demonstrates polyglot microservices architecture.
- **Kubernetes Deployment:** Migrate from Docker Compose to Kubernetes (minikube) with Helm charts, horizontal pod autoscaling, and rolling deployments.
- **GraphRAG Integration:** Combine TigerGraph knowledge graph with LLM retrieval-augmented generation for explainable fraud decisions — a cutting-edge approach adopted by leading financial institutions.
- **Event Sourcing:** Implement event sourcing for the payments service to maintain a complete audit trail of all state changes — critical for financial compliance.
- **Multi-currency Support:** Add Wise API integration for cross-border payments with real-time currency conversion.